DP-OPS-0016: Governance Hardening and TASK.md Template Refactor

1. Operational Context and State Forensic Analysis

1.1. System State and Freshness Authentication

The initiation of **DP-OPS-0016** requires a rigorous establishment of the repository's current state. This is not merely a procedural step but a forensic necessity to ensure that the proposed governance hardening measures are applied to a deterministic and authenticated baseline. The analysis of the provided OPEN artifact and repository dump confirms the active operational context.

The active branch is identified as main, with the HEAD hash currently residing at 4ad69a0e7.[1] This hash represents the most recent immutable state of the Stela platform, specifically the merge of pull request #133, which involved updates to the tasks sub-system.[1] The working tree is reported as "clean," with zero porcelain entries and no saved porcelain artifacts, indicating a quiescent state ready for strictly controlled modification.[1]

This state serves as the "Base HEAD" for the Dispatch Packet (DP). Adherence to this freshness gate is critical; any drift from 4ad69a0e7 would invalidate the assumptions regarding the content of TASK.md and the behavior of the policing workflows. The OPEN prompt explicitly instructs the Operator to "Adhere to the Behavioral Logic Standards defined in AGENTS.md" and to "Follow repo canon".[1] This creates a recursive governance requirement: the mechanism used to update the governance structures (the DP) must itself strictly adhere to the governance structures it seeks to modify.

The "Next Operator Moves" indicated in the OPEN prompt point to executing a platform-scope dump: ./ops/bin/dump --scope=platform --format=chatgpt.[1] This command, which excludes the projects/ directory, provides the necessary visibility into the ops/, docs/, and tools/ directories where the target surfaces for this operation reside. The dump manifest confirms that 73 files were included in the scope, capturing the full operational skeleton of the system.[1] This comprehensive snapshot allows for a granular analysis of the friction points identified in the user query.

1.2. Problem Definition: The Context Pollution Vector

The primary friction point necessitating this intervention is the phenomenon of "Context Pollution" affecting AI workers. The user query highlights a recurring failure mode: "TASK/DP WRITER is still adding read in for OPEN AT BEGINNING FOR WORKER." This practice, while seemingly benign, represents a fundamental violation of the "Role Contract" defined in OPEN.[1]

The OPEN artifact is designed exclusively for the **Integrator (Operator)**. It contains the "Freshness Gate," "Role Contract," and high-level "Constitution" pointers intended to orient the human operator or the high-level planning agent.[1] It sets the "Collaborator Stance" and defines what the operator must *do* (e.g., "PR-only changes," "Run required gates").

When this artifact is loaded into the context of a **Contractor (Worker)**, it introduces semantic noise. The worker, whose role is "Executes tasks, flags risks, asks when blocked" [1], does not need to know about the "Next Operator Moves" or the "Freshness Gate" verification steps that occurred *before* the DP was assigned. Ingesting OPEN consumes valuable token bandwidth—often 1,000 to 2,000 tokens depending on the verbosity of the git status—and risks confusing the worker regarding its jurisdiction. A worker reading "Integrator (operator): Owns scope" might hallucinate that it *is* the operator if the prompt structure is ambiguous.

Furthermore, the user query notes that "DP WRITER IS CONSISTENTLY adding artifacts from CHAT conversation FOR WORKER." This violates the TRUTH.md filing doctrine, which explicitly categorizes "TEMPORARY trash" to the storage/ directory.[1] Chat logs, reasoning traces, and intermediate conversation artifacts are ephemeral. Embedding them in the canonical context of a DP creates "Ghost Canon"—rules or context that exist only in the chat history and not in the repository files. This degrades the "My Local is Truth" constraint defined in TASK.md.[1]

1.3. The Receipt Gap and Audit Readiness

The second critical failure mode identified is the inconsistency of closeout receipts. The user query states: "closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN."

In the Stela governance model, a completed task is not defined merely by the code change, but by the **Proof Bundle**. This bundle acts as a forensic chain of custody. The "Porcelain" artifact (OPEN-PORCELAIN) proves the cleanliness of the workspace *before* work began. The "Open" artifact (OPEN) proves the intent was captured and authenticated against the git state. The "Dump" artifact (dump-full...) provides a serialized snapshot of the final state *after* the work is complete.

Currently, TASK.md (as referenced in AGENTS.md and the policing workflows) likely encourages these receipts but lacks a "Hard Gate" to enforce them. Without strict enforcement, workers—optimizing for speed or minimizing output—often skip the dump or fail to save the porcelain state. This leaves the repository history vulnerable. If a regression occurs, there is no artifact proving that the environment was clean when the change was introduced.

1.4. Semantic Flattening in Metadata

The third driver for DP-OPS-0016 is the degradation of metadata quality in git history. The

user query observes a need to "VARY THE WORDING EVERY TEXT FIELD" in the closing block.

Current behaviors suggest a pattern of "Semantic Flattening," where the same text string is used for the Git Commit Message, the Pull Request Title, and the Pull Request Description. While efficient, this collapses the semantic dimensionality of the change log.

- The **Commit Message** should be technical and imperative (e.g., "fix: strict context boundaries in TASK.md").
- The **PR Title** should be descriptive and human-readable (e.g., "Governance: Hardening TASK.md Context and Closeout Protocols").
- The **PR Description** should be contextual, explaining the *why* and *how*.

By forcing the worker to generate different text for each field (e.g., "1. Commit message" vs "4. Commit message"), we introduce "Cognitive Friction." This psychological design pattern forces the agent to re-evaluate the content for each field, preventing the lazy replication of a single summary string and ensuring that the repository history contains rich, multi-faceted metadata.[2]

---

2. Architectural Design of the Governance Upgrade

2.1. Refactoring TASK.md Section I: Context Boundaries

The first architectural change targets **Section I (REQUIRED CONTEXT LOAD)** of the TASK.md template. The objective is to enforce a "Zero-Trust" context model for the worker.

- **Current State:** The template likely lists OPEN as a required artifact, or at least allows it implicitly.
- **New State:** The allowlist for context loading will be strictly defined as: TRUTH, AGENTS, SoP, CONTEXT, and MAP.
- **Negative Constraint:** A new directive will be added: (OPEN is for Planner/DP Writer state refresh; Worker does not read OPEN). This explicit exclusion instructs the Large Language Model (LLM) to ignore or reject OPEN artifacts if they are accidentally provided, reinforcing the jurisdictional boundary between Operator and Contractor.[1]

2.2. Establishing the Forensic Triad in Section E

To address the "Receipt Gap," we will formalize the "Forensic Triad" within **Section E (RECEIPT)** of the template. This goes beyond a simple checklist; it establishes a **Definition of Done**.

The template will be updated to explicitly list the required artifacts with their standard naming conventions:

1. storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt: The pre-state fingerprint.
2. storage/handoff/OPEN-[work-branch]-[hash].txt: The intent fingerprint.

3. storage/dumps/dump-full-...: The post-state serialization.

A "MANDATE" clause will be added: **"No receipt/proof bundle = AUTOMATIC DISAPPROVAL."** This language invokes the "Absolute Literalism" behavior defined in AGENTS.md [1], forcing the AI agent to prioritize receipt generation above all other closeout activities.

2.3. The Mandatory Closing Block Schema

The most significant structural addition is the **New Mandatory Closing Block** in Section 3. The user's requirement to "vary the wording" for identical fields requires a careful semantic design. We cannot simply repeat "Commit message" four times, as this invites redundancy. Instead, we must use synonyms that imply different audiences or granularities.[3]

We will derive the specific field labels from industry best practices for git metadata and form design:

1. **Commit message (plaintext)** → Label: **"1. Primary Commit Header (plaintext)"**
   - *Intent:* The subject line of the first commit. Concise, imperative (e.g., "refactor: update TASK.md context rules").[5]
2. **Add a title (plaintext)** → Label: **"2. Pull Request Title (plaintext)"**
   - *Intent:* The headline for the PR review. Descriptive, context-rich (e.g., "Governance Hardening: TASK.md Context Hygiene and Receipt Enforcement").[7]
3. **Add a description (markdown)** → Label: **"3. Pull Request Description (markdown)"**
   - *Intent:* The body of the PR. A detailed narrative of changes, rationale, and risks.[8]
4. **Commit message (plaintext)** → Label: **"4. Final Squash Stub (plaintext)"**
   - *Intent:* The subject line for the final squashed commit. This often differs from the primary header if the scope expanded during work. It serves the permanent history.[9]
5. **Extended description (plaintext)** → Label: **"5. Extended Technical Manifest (plaintext)"**
   - *Intent:* The detailed body of the git commit message. This should list specific file changes and technical details, distinct from the broader PR description.[10]
6. **Add a comment (markdown)** → Label: **"6. Review Conversation Starter (markdown)"**
   - *Intent:* A human-centric comment added to the PR to guide the reviewer or highlight specific areas for attention.[12]

This schema satisfies the user's "Same but Different" constraint by assigning distinct semantic roles to each field, preventing the agent from simply copying and pasting a single string.

2.4. Allowlist Recursion and Branching

The execution of DP-OPS-0016 requires modifying TASK.md itself. This creates a meta-level requirement: TASK.md must be in the Target Files allowlist of the DP that modifies it. Additionally, because TASK.md is a "Canon" surface (changing the rules of the system), SoP.md must also be included in the allowlist to record the state change, per sop_policing.yml.[1]

- **Branch Name:** work/task-governance-update-2026-02-03
  - This adheres to the work/<topic>-YYYY-MM-DD convention found in CONTRIBUTING.md.[1]
- **Allowlist:**
  - TASK.md
  - SoP.md

---

3. Detailed Analysis of Research Findings

3.1. The "Varied Wording" Hypothesis

Research into form design best practices supports the user's directive to vary wording. Studies indicate that "Label blindness" occurs when users (or agents) encounter repetitive fields.[2] By differentiating "Commit message" (Field 1) from "Commit message" (Field 4) via distinct labels like "Primary Commit Header" and "Final Squash Stub," we force a "re-reading" event.

Snippet [3] suggests using the "imperative mood" for commit subjects (e.g., "Add unit tests").[3] Snippet [7] advises that PR titles should be "concise and specific" and include identifiers.[7] Snippet [11] highlights that git commit messages can have a "subject" and a "body" separated by a newline.[11] By explicitly requesting an "Extended Technical Manifest" (Field 5), we enforce the creation of this "body" content, which is often omitted in automated workflows.

3.2. Receipt Generation Forensics

The ops/bin/dump script [1] reveals that it supports a --bundle flag which generates a "Dump Manifest" alongside the payload. This manifest contains critical metadata: Branch, HEAD, Scope, and Included Files.[1] The ops/bin/open script [1] generates the OPEN prompt and saves it to storage/handoff/.

The inconsistency noted by the user ("receipts not being produced CONSISTENTLY") suggests that workers are running these commands but not *persisting* or *grouping* the outputs. The TASK.md update must therefore not just ask for the *actions* (Run dump, Run open) but for the *artifacts* (Check for file X, Check for file Y). This shifts the validation from "Did you run it?" (Behavioral) to "Does the file exist?" (Existential), which is verifiable by the ls command in the

results block.

3.3. Context Loading and Token Economics

The OPEN prompt typically contains a "Freshness Gate" with the current git status. If the workspace is dirty, ops/bin/open prints the git status --porcelain output.[1] For a large repo, this can be hundreds of lines. Loading this into the worker's context is wasteful. The worker only needs to know *what to do* (The DP), not the *history of how we got here* (The OPEN prompt).

By enforcing the negative constraint "Worker does not read OPEN," we free up context window space for the actual code files (TASK.md, SoP.md) and the relevant library documentation. This improves the worker's "attention" on the task at hand.

---

4. Execution Specification: DP-OPS-0016

The following section constitutes the executable Dispatch Packet. It is drafted strictly according to the TASK.md headings (as inferred from dp_lint.sh and AGENTS.md) and incorporates all research findings and user directives.

4.1. Meta-Data Configuration

- **DP ID:** DP-OPS-0016
- **Title:** TASK.md Governance Hardening & Context Hygiene
- **Base Branch:** main
- **Base HEAD:** 4ad69a0e7
- **Work Branch:** work/task-governance-update-2026-02-03

4.2. Target Surface Definitions

The DP targets TASK.md for refactoring. It specifically mandates the insertion of the "MANDATORY CLOSING BLOCK" and the revision of the "REQUIRED CONTEXT LOAD" section. It also targets SoP.md to log this governance change.

4.3. The Dispatch Packet

DP-OPS-0016: TASK.md Governance Hardening & Context Hygiene

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: main

Required Work Branch: work/task-governance-update-2026-02-03

Base HEAD: 4ad69a0e7

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - TASK.md (Target surface)

Negative Constraint (Strict):

- Do NOT load OPEN. (OPEN is for Planner/DP Writer state refresh only).
- Do NOT load chat logs or reasoning traces. (These are disposable artifacts).

II) SCOPE & SAFETY

- **Objective:** Refactor TASK.md to enforce strict context boundaries (no OPEN), mandate consistent receipt generation, and implement a rigorous, varied-wording closing block for commit metadata.
- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin binaries.
  - No changes to docs/ library content.

Target Files allowlist (hard gate)

- TASK.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The current TASK.md template inadvertently encourages context pollution by including OPEN in the worker's load list. It also lacks a rigid structure for closeout metadata, leading to identical/low-quality commit messages and PR descriptions. Receipt generation (Porcelain/Dump/Open) is inconsistent.
- **Drift:** Workers are consuming tokens on operator-only context (OPEN) and failing to produce forensic proof bundles. Git history is suffering from "Semantic Flattening" due to repetitive metadata fields.
- **Desired State:** TASK.md strictly excludes OPEN from worker context. A new "Mandatory Closing Block" enforces distinct, varied wording for all commit/PR fields. The receipt checklist is expanded to mandate specific artifact paths.

B) REQUEST

Numbered tasks (concrete, no vibes):

1. **Refactor TASK.md Section 0 & I:**
    - Remove OPEN from the worker's required reading list.
    - Insert the instruction: (OPEN is for Planner/DP Writer state refresh; Worker does not read OPEN).
    - Add the negative constraint: DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.
2. **Refactor TASK.md Section 2 (CURRENT DP) - Subsection E (RECEIPT):**
    - Expand the "Proof bundle checklist" to explicitly list:
        - storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt
        - storage/handoff/OPEN-[work-branch]-[hash].txt
        - storage/dumps/dump-full-... (or platform scope as applicable)
    - Add the mandate: closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.
3. **Refactor TASK.md Section 3 (CLOSEOUT):**
    - **Insert the NEW MANDATORY CLOSING BLOCK.**
    - This block must follow the "Varied Wording" provision. Use the following exact schema to enforce differentiation:
        1. 1. Primary Commit Header (plaintext)
        2. 2. Pull Request Title (plaintext)
        3. 3. Pull Request Description (markdown)
        4. 4. Final Squash Stub (plaintext) (Must differ from #1)
        5. 5. Extended Technical Manifest (plaintext)
        6. 6. Review Conversation Starter (markdown)
4. **Update SoP.md:**
    - Append a new entry logging the governance hardening: DP-OPS-0016: Hardened TASK.md (context hygiene + mandatory closing block).

Required Content (optional; use when doc/governance changes demand exact text/format):

- The "Mandatory Closing Block" must explicitly use the 6 varied labels defined in Task #3 to prevent auto-pilot repetition.

C) CHANGELOG

- **Log (human-readable, 1–6 bullets):**
    - Removed OPEN from Worker context load in TASK.md.
    - Added "Mandatory Closing Block" with 6 distinct metadata fields (Varied Wording enforced).
    - Mandated OPEN, PORCELAIN, and DUMP artifacts in Receipt checklist.
    - Added negative constraint against chat/disposable artifact inclusion.
    - Updated SoP.md to reflect canon governance change.

D) PATCH / DIFF

- **Format:** Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- Context lint (use repo-canonical command):
  - Prefer: bash tools/context_lint.sh
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint_truth.sh

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/skill/skill_lib.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/. Refine Invocation guidance and Solution. Do not edit the Provenance block.
- Promote: run ops/lib/skill/skill_lib.sh promote <draft_path> or log "no new skill promoted" in RESULTS.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path.
  - Promotion: command invoked plus grep -n "Promotion Packet:" SKILL.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0016-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-*.txt (required)
- storage/handoff/OPEN-*.txt (required)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)

- Verification: (Output of proofs)

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0016-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied.
- [ ] Fill out the **MANDATORY CLOSING BLOCK** (see Section III.B).
- [ ] Append a **TASK Work Log** entry.

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

3.1) THREAD TRANSITION (RESET / ARCHIVE RULE)

See TASK.md standard.

---

4) WORK LOG (TIMESTAMPED CONTINUITY)

- *2026-02-03 11:08* — DP-OPS-0016: Generated Governance Hardening DP. Verification: PENDING. Blockers: none. NEXT: Dispatch to worker.

---

5. Strategic Justification and Second-Order Effects

5.1. The "Cognitive Friction" of Varied Wording

The requirement to vary the wording of the closing block fields is a deliberate application of "Cognitive Friction" in user interface design.[13] When a user (or an advanced AI agent) encounters repetitive fields like "Commit Message," "Commit Message," "Commit Message," the brain (or the model's predictive attention) disengages, leading to the "Default Value" error—where the same string is pasted into all fields.[14]

By relabeling these fields to "Primary Commit Header," "Pull Request Title," and "Final Squash

Stub," we force the agent to process the semantic nuance of each field.

- **Primary Commit Header:** Suggests brevity and immediate action (e.g., fix: update TASK context).
- **Pull Request Title:** Suggests a broader scope for human review (e.g., Governance: Hardening Context Boundaries).
- **Final Squash Stub:** Suggests the permanent historical record (e.g., DP-OPS-0016: Context Hygiene).

This simple linguistic variation significantly increases the metadata quality of the repository history, making future audits and regressions easier to trace.[15]

5.2. Context Hygiene as a Security and Efficiency Vector

Removing OPEN from the worker's context load is an efficiency optimization that doubles as a security measure.

- **Efficiency:** The OPEN prompt contains the entire git status. In a large repo, this is noise. Removing it saves tokens.
- **Security:** OPEN contains Operator-level instructions ("Next Operator Moves"). By hiding this from the Worker, we implement the "Principle of Least Privilege" for context. The Worker cannot be confused by instructions meant for the Operator.[16]

5.3. The Receipt Mandate and "Definition of Done"

The explicit listing of OPEN-PORCELAIN, OPEN, and DUMP artifacts moves the "Definition of Done" from a subjective assessment ("Did you finish?") to an objective one ("Are these files present?"). This is crucial for autonomous or semi-autonomous workflows. If the file is missing, the task is failed. This binary success condition allows for automated policing and prevents "drift" where workers gradually stop performing cleanup tasks.[17]

6. Conclusion and Recommendation

DP-OPS-0016 is a critical infrastructure upgrade. It hardens the TASK.md template against the specific drift vectors identified in recent operations: context pollution, receipt negligence, and metadata flattening.

By enforcing strict context boundaries and implementing the "Varied Wording" closing block, we transform TASK.md from a passive template into an active governance enforcement tool. This ensures that the Stela repository maintains its high standards of provenance and auditability even as the velocity of AI-driven changes increases.

Recommendation: Proceed immediately with the execution of DP-OPS-0016 on the designated branch work/task-governance-update-2026-02-03. Verify the presence of the new Closing Block and the exclusion of OPEN from the context load before merging.

Works cited

1. OPEN-main-4ad69a0e7.txt
2. Form design best practices by Andrew Coyle, accessed February 3, 2026, https://www.andrewcoyle.com/blog/form-design-best-practices
3. The Art of Writing Meaningful Git Commit Messages | by theurikarue - Medium, accessed February 3, 2026, https://medium.com/@iambonitheuri/the-art-of-writing-meaningful-git-commit-messages-a56887a4cb49
4. What are the various labels your team or project uses in GitHub or otherwise?, accessed February 3, 2026, https://dev.to/ben/what-are-the-various-labels-your-team-or-project-uses-in-github-or-otherwise-22em
5. Commit message guidelines - GitHub Gist, accessed February 3, 2026, https://gist.github.com/robertpainsi/b632364184e70900af4ab688decf6f53
6. Git Commit Message Standard - GitHub Gist, accessed February 3, 2026, https://gist.github.com/turbo/efb8d57c145e00dc38907f9526b60f17
7. Best practices for writing clear pull request titles and descriptions - Graphite, accessed February 3, 2026, https://graphite.com/guides/best-practices-pr-titles-descriptions
8. How to Write an Awesome Pull Request (PR) Description | by Julio Lugo | Medium, accessed February 3, 2026, https://medium.com/@jmanuellugo96/how-to-write-an-awesome-pull-request-pr-description-bdd2c6e48418
9. A Ruleset with Metadata Restrictions on Commit Messages should not block Squash Merge Pull Requests unless the sqash commit message itself violates the rule · community · Discussion #108531 - GitHub, accessed February 3, 2026, https://github.com/orgs/community/discussions/108531
10. The art of writing meaningful Git commit messages - Thoughtbot, accessed February 3, 2026, https://thoughtbot.com/blog/the-art-of-writing-meaningful-git-commit-messages
11. How to do a git commit with a subject line and message body? - Stack Overflow, accessed February 3, 2026, https://stackoverflow.com/questions/40505643/how-to-do-a-git-commit-with-a-subject-line-and-message-body
12. Commenting on a pull request - GitHub Docs, accessed February 3, 2026, https://docs.github.com/articles/commenting-on-a-pull-request
13. Best Practices | Dynamic Forms - Yale University, accessed February 3, 2026, https://forms.sis.yale.edu/designer-resources/best-practices
14. Required Fields in Forms: Best Design Practices - UX Tigers, accessed February 3, 2026, https://www.uxtigers.com/post/required-fields
15. Git commit message best practices, accessed February 3, 2026, https://graphite.com/guides/git-commit-message-best-practices
16. Conventional Commits, accessed February 3, 2026,

https://www.conventionalcommits.org/en/v1.0.0/
17. Writing good commit messages and enforcing valid Pull Request titles on GitHub - Satellytes, accessed February 3, 2026, https://www.satellytes.com/blog/post/writing-and-enforcing-conventional-commit-messages-and-pull-request-titles/